Dynamic Memory, Functions and Object Oriented Concepts

Dr. Mohammad Maksood Akhter
Assistant Professor

Department of Computer Science and Engineering

Feburary 25, 2026

Motivation for Dynamic Memory Allocation

In many programs, the amount of memory required is not known at compile time.
Example:

- Suppose we want to store marks of students.
- The number of students may vary every time the program runs.

If we use static arrays:

```
int marks[100];
```

Problems:

- If there are only 20 students, most of the memory remains unused.
- If there are 150 students, the array size becomes insufficient.

Therefore, static allocation is inefficient in many real applications.
Dynamic memory allocation solves this problem by allowing memory to be created during program execution.

What is Dynamic Memory Allocation?

Dynamic Memory Allocation is the process of allocating memory during the execution of a program.

In C++, this is done using the following operators:

- **new** – used to allocate memory dynamically
- **delete** – used to release allocated memory

Characteristics:

- Memory is allocated from the **heap**.
- The size of memory can be decided at runtime.
- It provides flexibility in managing memory efficiently.

Example applications:

- Dynamic arrays
- Linked lists
- Trees and graphs

Stack vs Heap Memory

In a running program, memory is divided into different regions.

Stack Memory

- Stores local variables of functions
- Memory allocation and deallocation happen automatically
- Managed by the compiler
- Faster access
- Limited memory size

Heap Memory

- Used for dynamic memory allocation
- Memory is allocated using **new**
- Memory must be released using **delete**
- Managed by the programmer
- Larger memory space but slower than stack

Memory Layout of a Program

Typical memory layout of a running program:

- **Code Segment** – stores program instructions
- **Data Segment** – stores global and static variables
- **Heap** – used for dynamic memory allocation
- **Stack** – stores function calls and local variables

Important observation:

- Stack grows downward
- Heap grows upward

Dynamic memory allocation occurs in the heap region.

Why Dynamic Memory is Important

Dynamic memory allocation is important because:

- Program memory requirements may change during execution
- Efficient memory utilization
- Supports advanced data structures

Many important data structures rely on dynamic allocation:

- Linked Lists
- Binary Trees
- Graph Structures
- Dynamic Arrays

Therefore, understanding dynamic memory allocation is essential for building scalable software systems.

Using the new Operator

In C++, dynamic memory is allocated using the **new** operator.
General syntax:

```
        pointer = new data_type;
```

Example:

```
        int *p;
        p = new int;
```

Explanation:

- Memory for one integer is allocated in the **heap**.
- The address of the allocated memory is returned by **new**.
- This address is stored in pointer **p**.

Now the pointer **p** can be used to access the dynamically allocated memory.

Initialization during Allocation

Memory can also be initialized at the time of allocation.
Example:

```
int *p = new int(10);
```

Explanation:

- Memory is allocated dynamically for an integer.
- The allocated memory is initialized with value **10**.
- Pointer **p** stores the address of that memory location.

Accessing the value:

```
cout << *p;
```

Output: **10**

Dynamic Allocation of Arrays

We can allocate memory for arrays dynamically using the **new[]** operator.
Example:

```
int *arr = new int[5];
```

Explanation:

- Memory for **5 integers** is allocated in the heap.
- Pointer **arr** stores the address of the first element.
- The array elements can be accessed using indexing.

Example:

```
arr[0] = 10;
arr[1] = 20;
arr[2] = 30;
```

Releasing Dynamically Allocated Memory

Memory allocated using **new** must be released using the **delete** operator.
Syntax:

```
delete pointer;
```

Example:

```
delete p;
```

For arrays:

```
delete[] arr;
```

Explanation:

- **delete** releases memory allocated for a single variable.
- **delete[]** releases memory allocated for arrays.
- This returns memory back to the heap for reuse.

Memory Leak

A **memory leak** occurs when dynamically allocated memory is not released.
Example:

- Memory is allocated using **new**.
- The program finishes without calling **delete**.

Consequences:

- Memory remains occupied even though it is no longer used.
- Available memory gradually decreases.
- Long-running programs may eventually crash.

Therefore, every **new** operation should have a corresponding **delete**.

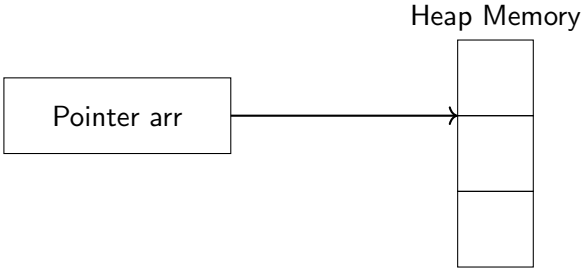
Complete Example: Dynamic Array

Example program that dynamically allocates memory for an array.

```
#include<iostream>
using namespace std;

int main()
{
    int n;
    cin >> n;

    int *arr = new int[n];

    for(int i=0;i<n;i++)
    cin >> arr[i];

    delete[] arr;

    return 0;
}
```

Memory for the array is allocated at runtime based on user input.

Pointer and Heap Memory Representation



- Pointer stores the address of dynamically allocated memory.
- Actual data is stored in heap memory.

Dangling Pointer

A **dangling pointer** is a pointer that points to memory that has already been released.
Example:

```
int *p = new int(10);

delete p;

cout << *p;  // Dangerous
```

Explanation:

- Memory is released using delete.
- Pointer p still holds the old address.
- Accessing it may cause unpredictable behaviour.

Solution:

```
p = nullptr;
```

Difference Between malloc and new

Feature	malloc	new
Language	C	C++
Return Type	void*	Correct pointer type
Initialization	No	Yes
Constructor Call	No	Yes
Memory Release	free()	delete

Example:

- malloc() is used in C programming.
- new is the preferred method in C++.

Concept Questions

Question 1

Why is dynamic memory allocation required in programming?

Concept Questions

Question 1

Why is dynamic memory allocation required in programming?

Question 2

What is the difference between stack memory and heap memory?

Concept Questions

Question 1

Why is dynamic memory allocation required in programming?

Question 2

What is the difference between stack memory and heap memory?

Question 3

Which operator is used to allocate dynamic memory in C++?

Concept Questions

Question 1

Why is dynamic memory allocation required in programming?

Question 2

What is the difference between stack memory and heap memory?

Question 3

Which operator is used to allocate dynamic memory in C++?

Question 4

What happens if dynamically allocated memory is not released?

Concept Questions

Question 1

Why is dynamic memory allocation required in programming?

Question 2

What is the difference between stack memory and heap memory?

Question 3

Which operator is used to allocate dynamic memory in C++?

Question 4

What happens if dynamically allocated memory is not released?

Question 5

What is a dangling pointer?

Quick Quiz

1. Which operator allocates dynamic memory in C++?
A. malloc B. new C. alloc D. create

2. Which operator releases dynamically allocated memory?
A. free B. release C. delete D. remove

Motivation for Using Functions

Large programs become difficult to manage if written as a single block of code.
Functions help in dividing a program into smaller and manageable units.
Advantages of using functions:

- **Code Reusability** – A function can be used multiple times in a program.
- **Improved Readability** – Programs become easier to understand.
- **Easier Debugging** – Errors can be isolated within specific functions.
- **Modular Programming** – Programs can be developed in logical modules.

Example idea:
Instead of repeating the same code many times, we define a function once and reuse it.

Basic Structure of a Function

A function generally consists of three main parts:

- **Function Declaration** – Tells the compiler about the function.
- **Function Definition** – Contains the actual implementation.
- **Function Call** – Invokes the function during program execution.

Example flow:

$$\text{Main Program} \rightarrow \text{Function Call} \rightarrow \text{Function Execution} \rightarrow \text{Return Result}$$

Function Syntax

General syntax of a function in C++:

```
return_type function_name(parameters)
{
 statements;
 return value;
}
```

Explanation:

- **return_type** – Type of value returned by the function.
- **function_name** – Name used to call the function.
- **parameters** – Inputs passed to the function.

Example Function

Example of a function that adds two numbers.

```
int add(int a, int b)
{
    return a + b;
}
```

Explanation:

- The function takes two integers as input.
- It returns the sum of these integers.
- The return type is **int**.

Function Call

Functions are executed when they are called.
Example:

```
        int result;

        result = add(5,6);
```

Explanation:

- The values 5 and 6 are passed as arguments.
- The function computes the sum.
- The returned value is stored in variable **result**.

Types of Functions

Functions can be categorized based on parameters and return values.

1. **No Parameters and No Return Value**
2. **Parameters but No Return Value**
3. **Parameters with Return Value**
4. **No Parameters but Return Value**

These categories help us design functions based on program requirements.

Pass by Value

In pass by value, copies of the variables are passed to the function.

```
void swap(int a, int b)
{
    int temp = a;
    a = b;
    b = temp;
}
```

Important observation:

- Changes made inside the function do not affect original variables.
- Only copies of the values are modified.

Pass by Reference

In pass by reference, the function receives references to the original variables.

```
void swap(int &a, int &b)
{
    int temp = a;
    a = b;
    b = temp;
}
```

Explanation:

- The symbol **&** represents reference.
- Changes made inside the function directly affect the original variables.
- This avoids unnecessary copying of data.

Inline Functions

Inline functions are used to reduce the overhead of function calls.
Example:

```
inline int square(int x)
{
    return x * x;
}
```

Explanation:

- The compiler replaces the function call with the function body.
- Useful for small functions.
- Improves program performance.

Inline Functions

```
inline void displayNum(int num) {
    cout << num << endl;
}
int main() {
   displayNum(5);
   displayNum(8);
   displayNum(666);
}
```

Compilation →

```
inline void displayNum(int num) {
    cout << num << endl;
}
int main() {
    cout <<< 5 << endl;
    cout << 8 << endl;
    cout << 666 << endl;
}
```

Default Arguments

C++ allows functions to have default parameter values.
Example:

```
int sum(int a, int b = 5)
{
    return a + b;
}
```

Usage:

```
sum(10);    // returns 15
sum(10,3); // returns 13
```

Explanation:
If the second argument is not provided, the default value is used.

Function Execution Flow

Program execution follows this sequence:

Main Function

$\downarrow$

Function Call

$\downarrow$

Function Execution

$\downarrow$

Return Value

This mechanism enables modular program design.

Object Oriented Programming

Object Oriented Programming (OOP) is a programming paradigm that organizes programs around objects rather than functions.
In OOP, real-world entities are modeled using classes and objects.
Example real-world analogy:

- Class : Student
- Objects : Alice, Bob, John

Key principles of OOP:

- **Encapsulation** – Bundling data and functions together
- **Abstraction** – Hiding unnecessary implementation details
- **Reusability** – Code can be reused through classes
- **Modularity** – Programs are divided into independent units

Concept of Class and Object

A **Class** is a user-defined data type that represents a group of related data and functions.
It acts as a blueprint or template for creating objects.

Object

An object is an instance of a class.
Each object contains its own copy of the class data members.
Example:

- Class : Car
- Objects : BMW, Audi, Tesla

Thus,

$$\text{Class} \rightarrow \text{Blueprint}$$
$$\text{Object} \rightarrow \text{Real instance}$$

Basic Syntax of a Class

General structure of a class in C++:

```
class ClassName
{
    access_specifier:

    data_members;

    member_functions;
};
```

Important points:

- The keyword **class** is used to define a class.
- Data members store information.
- Member functions operate on the data members.
- A semicolon is required at the end of the class definition.

Creating Objects

Objects are created from a class just like variables are created from a data type.
Example:

```
Student s1;
Student s2;
```

Explanation:

- **s1** and **s2** are objects of the class Student.
- Each object has its own copy of the data members.

Accessing Data Members

Data members of an object are accessed using the dot operator (.).
Example:

```
Student s1;

s1.id = 10;
s1.marks = 90;
```

Explanation:

- The dot operator connects the object with its members.
- s1.id refers to the id of object s1.

Access Specifiers in C++

Access specifiers control the visibility of class members.
Three types of access specifiers:

- **public**
  Members are accessible from outside the class.
- **private**
  Members are accessible only within the class.
- **protected**
  Members are accessible within the class and derived classes.

Default Access Specifier

In C++, the default access specifier depends on the keyword used.

- For **class** → default access is **private**
- For **struct** → default access is **public**

Example:

- class Example → members are private by default
- struct Example → members are public by default

Member Functions

Member functions define the behavior of objects.
They operate on the data members of the class.
Example:

```
class Student
{
    public:
    int id;

    void display()
    {
        cout << id;
    }
};
```

Explanation:

- **display()** is a member function.
- It prints the value of the data member id.

Complete Example Program

```
#include<iostream>
using namespace std;
class Student
{   public:
    int id;
    float marks;
    void display()              {
        cout << id << " " << marks;
    }
};
int main()
{   Student s1;
    s1.id = 10;
    s1.marks = 95;
    s1.display();
}
```

Programming Exercise

Write a C++ program that:

- Creates a class **Student**
- Stores student ID and marks
- Defines a member function to display the information
- Creates two objects of the class

Constructors in C++

A constructor is a special member function of a class.
It is automatically called when an object of the class is created.
Purpose of constructor:

- Initialize object data members
- Allocate resources if needed
- Prepare the object for use

Important properties:

- Constructor name must be the same as the class name
- Constructors have no return type
- Constructors are automatically invoked

Default Constructor Example

A constructor with no parameters is called a **default constructor**.

```
#include<iostream>
using namespace std;

class Test
{
    public:
    Test()
    {
        cout << "Constructor called";
    }
};
```

Explanation:

- When an object of class Test is created
- The constructor is automatically executed

Creating Object and Constructor Call

```
int main()
{
    Test t1;
}
```

Output:
Constructor called
Explanation:

- Object t1 is created
- Constructor Test() is automatically executed

Parameterized Constructor

A constructor that accepts arguments is called a **parameterized constructor**.
It allows initialization of objects with specific values.

```
class Test
{
    public:
    Test(int x)
    {
        cout << x;
    }
};
```

Example:

```
Test t1(10);
```

Output:
10

Constructor for Data Initialization

Constructors are often used to initialize class variables.

```
class Student
{
    public:
    int id;

    Student(int x)
    {
        id = x;
    }
};
```

Usage:

```
Student s1(101);
```

Destructor in C++

A destructor is a special member function used to destroy objects.
It is automatically called when the object goes out of scope.
Purpose of destructor:

- Release memory resources
- Close files
- Free dynamically allocated memory

Important characteristics:

- Destructor name is same as class name
- It is preceded by symbol $\sim$
- It has no parameters
- Only one destructor per class

Destructor Syntax

```
class Test
{
    public:

    ~Test()
    {
        cout << "Destructor called";
    }

};
```

The destructor is executed automatically when the object is destroyed.

Complete Example of Constructor and Destructor

```
#include<iostream>
using namespace std;
class Test
{   public:
    Test()          {
        cout << "Constructor called" << endl;
    }
    ~Test()         {
        cout << "Destructor called" << endl;
    }
};
int main()
{   Test t1;
    return 0;
}
```

Output:
Constructor called
Destructor called

Programming Exercise

Write a C++ program that:

- Creates a class **Student**
- Uses a constructor to initialize student ID
- Displays the student information
- Uses a destructor to display a message when the object is destroyed

Introduction to Operator Overloading

Operator overloading allows existing operators to be redefined for user-defined data types such as classes.
This means operators like +, -, *, and == can work with objects.
Example:

- For integers: $5 + 3 = 8$
- For complex numbers: $(2 + 3i) + (4 + 5i)$

Without operator overloading, we would need to write a function such as:

- addComplex(c1, c2)

Operator overloading makes programs more readable and intuitive.

Introduction to Operator Overloading

Operator overloading allows existing operators to be redefined for user-defined data types such as classes.
This means operators like +, -, *, and == can work with objects.
Example:

- For integers: $5 + 3 = 8$
- For complex numbers: $(2 + 3i) + (4 + 5i)$

Without operator overloading, we would need to write a function such as:

- addComplex(c1, c2)

Operator overloading makes programs more readable and intuitive.

General Syntax of Operator Overloading

The keyword **operator** is used to define operator overloading.

```
        return_type operator symbol(parameters)
        {
            // statements
        }
```

Example:

```
        Complex operator+(Complex c)
        {
        }
```

Important points:

- Operator functions are member functions of a class
- The keyword operator is followed by the operator symbol
- Parameters represent the operands

Operators That Can Be Overloaded

Many operators in C++ can be overloaded.
Examples include:

- Arithmetic operators: +, -, *, /
- Relational operators: $==, !=, <, >$
- Increment and decrement: $++, -$
- Assignment operators

However, some operators cannot be overloaded:

- ::
- .
- sizeof
- ?:

Complex Number Example

Suppose we want to add two complex numbers using the + operator.

```
class Complex
{   int real, imag;
    public:
    Complex operator+(Complex c)            {
        Complex temp;
        temp.real = real + c.real;
        temp.imag = imag + c.imag;
        return temp;
    }
};
```

Using the Overloaded Operator

```
        Complex c1, c2, c3;

        c3 = c1 + c2;
```

Explanation:

- The + operator calls the overloaded operator function.
- Internally it behaves like:
  c3 = c1.operator+(c2);

Complete Example Program

```
class Complex       {
    int real, imag;
    public:
    void setData(int r, int i)          {
        real = r;
        imag = i;           }
    Complex operator+(Complex c)            {
        Complex temp;
        temp.real = real + c.real;
        temp.imag = imag + c.imag;
        return temp;            }
    void display()          {
        cout << real << "+" << imag << "i"; }
        };
int main()
{   Complex c1, c2, c3;
    c1.setData(2,3);
    c2.setData(4,5);
    c3 = c1 + c2;
    c3.display();       }
```

Type Conversion in C++

Type conversion refers to converting data from one type to another.
Two types of conversions exist in C++:

- Basic type to class type
- Class type to basic type
- Class type to class type

Example:
Converting an integer into a class object.

Basic Type to Class Type

Example:

```
class Distance
{
    int meter;

    public:

    Distance(int m)
    {
        meter = m;
    }
};
```

Usage:

```
Distance d = 10;
```

Here the integer value is converted into an object.

Class to Basic Type Conversion

Conversion from class object to basic data type.

Example:

```
class Test
{   int value;
    public:
    Test(int v)
    {
        value = v;
    }
    operator int()
    {
        return value;
    }
};
```

Usage:

```
Test t(5);
int x = t;
```

Programming Exercise

Write a C++ program that:

- Creates a class Complex
- Stores real and imaginary parts
- Overloads the + operator
- Adds two complex numbers
- Displays the result

Example:
$(2 + 3i) + (4 + 5i) = (6 + 8i)$

Questions ?

Thank You!